

GSoC21 W8: LFortran Refactors, and Compile Time Evaluations

Rohit Goswami · 2021-07-30 · gsoc21 · fortran · ramblings

Language standardese and implementations

Background

Serialized update for the 2021 Google Summer of Code under the fortran-lang organization, mentored by Ondrej Certik.

Logistics

- Discussed more refactors over MRs and Zulip

Overview

Intrinsic functions and more bug hunting. A lot of starts in different directions, but I will need to trim these down a bit. A major goal was working through the compile time evaluation of some intrinsic functions.

New Merge Requests

Split ast_to_asr

An MR started last week, completed and approved this week

tiny: Runtime implementation skeleton

What will eventually be compiled, hooks into `C` for now

tiny: Compile time implementation

The population of `value` for `tiny` function calls

Draft: Shift runtime intrinsic design

Harmonizing the code-base, much of this is cleaning up my own earlier math implementations

Draft: expr_value for Kind

A WIP MR which will clean up the slightly strange `extract_kind` function

Draft: Implement where construct

An MR along the lines of `if`, related but distinct from Gagandeep's masked optimization WIP

Freshly Assigned Issues

-show-asr For larger values

A visual glitch in the prettied output

Additional Tasks

Some of my earlier clean up MRs are beginning to stagnate (CI stuff), will have to catch up on them.

Misc Logs

The splitting of files for the refactor was harsh work. Took a few hours. Mindless, but really needs precision. Thankfully `vim` folds help a whole lot. This was further enhanced by Ondrej to make things even cleaner.

Intrinsic Design

I have discussed the design and implementation of these a few times before, but perhaps another write up will give direction to my thoughts. We have two major points of contact with the intrinsic functions:

Compile time

These are intrinsic functions like `tiny` or `kind` or even `sin` which can be evaluated immediately to populate the `expr_t* value` object

Runtime

These are the actual implementations, currently the goal is to have these hook into `C` libraries

Tiny Concerns

I am not a Fortran language lawyer, but I found myself puzzling over the legalese of the F-2018 draft standard with respect to the `C++` nearest neighbor, `std::numerical_limits`.

Essentially, the usage of `tiny` is meant to facilitate doing mathematics without worrying about the exact representability of the value; that is:

```
program main
  implicit none
  integer, parameter :: dp=kind(0.d0)
  real :: a=1.00000009
  if (abs(a-1)<tiny(1._dp)) then
    error stop "a-1 is effectively 0"
  end if
end program
```

Now by definition, `tiny` has the following properties:

1. **Description.** Smallest positive model number
2. **Class.** Inquiry function
3. **Argument.** X shall be a real scalar or array
4. **Result Characteristics.** Scalar with the same type and kind type parameter as X.
5. **Result Value.** The result has the value $b^{e_{min}-1}$ where b and e_{min} are defined in 16.4 for the model representing numbers of the same type and kind type parameter as X.
6. **Example.** TINY(X) has the value 2^{-127} for real X whose model is as in 16.4

This is fairly straightforward, once the model set for real x is understood as (from the section mentioned):

$$x = 0 || x = s \times b^e \times \sum_{k=1}^p f_k \times b^{-k}$$

Where b and p are integers exceeding one; each f_k is a nonnegative integer less than b , with f_1 nonzero; s is $+1$ or -1 ; and e is an integer that lies between the integer minimum and maxima. An extended model for real kinds relaxes the range of the exponent.

So far so good. The `C` equivalence is fairly straightforward, that is the `FLT_MIN` and `DBL_MIN` macros defined in `<float.h>`. This is infact what `gfortran` generates as well.

For a while though I was thrown by the fact that `C++`, within `<limits>` also has `std::numeric_limits<T>lowest()` (described [here](#)), which is smaller than the corresponding `min()` calls and has no direct `C` equivalent. It is infact, specifically mentioned to not be `min` for floating-point types.

However, I recognized soon enough from the implementation that there is no real conflict, as it is simply `-max()`, which says nothing about the representability.

Therefore, `tiny` must be `FLT_MIN` or `DBL_MIN`. I also took a small detour into `std::variant` before going with good old `if-else` early returns instead.

```
int tiny_kind = LFortran::ASRUtils::extract_kind_from_ttype_t(tiny_type);
if (tiny_kind == 4){
    float low_val = std::numeric_limits<float>::min();
    value = ASR::down_cast<ASR::expr_t>(ASR::make_ConstantReal_t(al, x.base.base.loc,
                                                                low_val, // value
                                                                tiny_type));
} else {
    double low_val = std::numeric_limits<double>::min();
    value = ASR::down_cast<ASR::expr_t>(ASR::make_ConstantReal_t(al, x.base.base.loc,
                                                                low_val, // value
                                                                tiny_type));
}
```

Conclusions

My thoughts have been turning towards scalability for a long time now. Good design before it is necessary is a premature optimization, but I think I would like to formulate a cleaner way of dealing with the intrinsic functions which can be reduced to values. The `ISO_C_BINDING` has been in my thoughts for a while now. Without it, runtime compilation of the ASR remains intractable. I expect the next week to continue along the same lines, populating value for all the intrinsic functions called by `minidftatom`. It would be best to also generate corresponding `runtime` implementations.